

SedSAT3

Command Line User's Manual

`sedstat3-cli`

Sediment Source Assessment Tool, version 1.1.6

October 1, 2026

Abstract

`sedsat3-cli` runs SedSAT3 analyses from a script instead of from the graphical interface. It loads a project, carries out a list of commands against it in order, and writes a report of what each one produced.

It is the same software. The command line build drives the same analysis code the windowed application drives, using the same command names and the same argument names, so anything that can be done through the dialogs can be written down in a script and repeated exactly.

This manual is for someone running analyses. It assumes familiarity with sediment fingerprinting and with SedSAT3 itself, but not with programming.

Contents

1	Introduction	3
1.1	What it is for	3
1.2	What it is not	3
1.3	How this manual is arranged	3
2	Installing	4
2.1	Requirements	4
2.2	Building	4
2.3	Telling it where the resources are	4
3	A first run	5
3.1	Seeing what is available	5
3.2	Writing a script	5
3.3	Running it	5
3.4	Reading the result	6
4	Scripts	7
4.1	Structure	7
4.2	Steps run in order, and they share one project	7
4.3	Steps	7
4.4	You need only give what you want to change	8
4.5	Paths	8
4.6	Checking before running	8
5	Configuring the project	9
5.1	set-target-group	9
5.2	set-elements	9
5.3	set-samples	10
5.4	set-element-role	10
6	Analysis commands	11
6.0.1	ANOVA	11
6.0.2	AutoSelect	11
6.0.3	BoxCox	12
6.0.4	Bracketing Analysis	12
6.0.5	Bracketing Analysis Batch	12
6.0.6	CMB Bayesian	12
6.0.7	CMB Bayesian-Batch	12
6.0.8	CorMat	13
6.0.9	CovMat	13
6.0.10	DF	13

6.0.11	DFA	13
6.0.12	DFAM	14
6.0.13	DFAOnevsRest	14
6.0.14	EDP	14
6.0.15	EDPM	15
6.0.16	Error_Analysis	15
6.0.17	GA	15
6.0.18	GA (disregarding targets)	15
6.0.19	GA (fixed elemental contribution)	16
6.0.20	KS	16
6.0.21	KS-individual	16
6.0.22	Levenberg-Marquardt	17
6.0.23	Levenberg-Marquardt-Batch	17
6.0.24	MLR	17
6.0.25	OM-Size Correct	17
6.0.26	Outlier	18
6.0.27	SDFA	18
6.0.28	SDFAM	18
6.0.29	SDFAOnevsRest	19
6.0.30	Source_Verify	19
6.0.31	Test CMB Bayesian	19
7	The report	20
7.1	What is in it	20
7.2	Results	20
7.3	Exit status	21
8	Options	22
9	Worked examples	23
9.1	A deterministic workflow	23
9.2	A Bayesian run	23
9.3	Comparing two tracer sets	24
10	When something goes wrong	25
10.1	The script was not run	25
10.2	A step failed	25
10.3	Nothing seems to be included in the analysis	25
10.4	Warnings about a singular system	25
10.5	It cannot find the command definitions	26

Chapter 1

Introduction

1.1 What it is for

The graphical interface is the right way to explore a dataset: you can see the plots, adjust a setting, and look again. It is a poor way to do the same thing twenty times, and it leaves no record of what was done.

The command line build addresses three things the interface cannot:

A run you can repeat

A script is a file. It states which project was loaded, which constituents were included, and which analyses were carried out with which settings. Running it again a year later produces the same thing, and anyone else can run it too.

A run you can vary

Comparing five tracer sets means five scripts differing in one line, not five sessions of clicking and remembering.

A run you can automate

It needs no display, so it can run on a server, in a scheduled job, or as part of a test that checks the results have not changed.

1.2 What it is not

It does not replace the interface. There are no plots, and there is no way to look at an intermediate result and change your mind. Use the interface to decide what to do, and a script to do it repeatedly.

One limitation is worth stating plainly at the outset: **the command line build cannot import from Excel**. It requires a project file that has already been saved from the interface. Importing a spreadsheet involves choosing which sheets to use and what each one holds, and those choices are made in dialogs that have no command line equivalent yet.

1.3 How this manual is arranged

Chapter 2 covers obtaining and building the program. Chapter 3 walks through a first run from beginning to end. Chapter 4 describes the script file in detail. Chapter 5 covers the commands that configure a project, and chapter 6 is the reference for every analysis command. Chapter 7 explains the report that a run produces, and chapter 10 covers what to do when something goes wrong.

Chapter 2

Installing

2.1 Requirements

The command line build needs the same libraries as the graphical one: Qt 6, the GNU Scientific Library, Armadillo, and a C++17 compiler. It links Qt Widgets but never opens a window, so it runs on a machine with no display.

2.2 Building

From the top of the source tree:

```
qmake6 SedSat3CLI.pro
make -j8
```

This produces `sedsat3-cli` in the build directory. On Windows, use the `qmake` supplied with the Qt 6 kit and the build tool that goes with your compiler; the project file is the same.

On systems with both Qt 5 and Qt 6 installed, the plain `qmake` command often resolves to Qt 5 and the build fails immediately. Use `qmake6`, or the full path to the Qt 6 one.

2.3 Telling it where the resources are

The program reads `resources/forms_structures.json`, which defines every command and its arguments. It uses that file to check a script before running it and to answer `-list-commands` and `-describe`.

It looks for the file next to the program and in the usual places relative to it. If your build directory is somewhere unusual, say where to look:

```
export SEDSAT3_RESOURCES=/path/to/SedSat3/resources
```

or on Windows:

```
set SEDSAT3_RESOURCES=C:\path\to\SedSat3\resources
```

The `-resources` option does the same thing for a single run.

If the file cannot be found the program still runs, but it warns you, and two things stop working: scripts are no longer checked, and arguments you leave out are no longer filled in with their defaults. The second of those matters more than it sounds, so it is worth setting the variable properly.

Chapter 3

A first run

3.1 Seeing what is available

```
sedst3-cli --list-commands
```

lists every command, and

```
sedst3-cli --describe "Levenberg-Marquardt"
```

prints one command's description and its arguments, with the default for each. These come from the same definitions the dialogs are built from, so they describe the software as it actually is.

3.2 Writing a script

A script is a text file containing JSON. Save the following as `run.json`, next to your project file:

```
{
  "project": "my-project.cmb",
  "output": "report.json",
  "steps": [
    {
      "command": "Levenberg-Marquardt",
      "arguments": {
        "Sample": "CTAIL1",
        "Apply size and organic matter correction": "false"
      }
    }
  ]
}
```

That is the whole thing: the project to load, where to put the report, and one analysis to carry out.

3.3 Running it

```
sedst3-cli run.json
```

You will see the step announced and its progress, and then where the report was written:

```
[1] Levenberg-Marquardt
    100%
report written to /home/you/work/report.json
```

3.4 Reading the result

The report is JSON. Each step records what was asked for, whether it succeeded, how long it took, and what it produced. The source contributions from the run above are under that step's results:

```
"1:Contributions": {
  "Bank":    0.2122,
  "Crop":    0.4104,
  "Forest":  0.1822,
  "Pasture": 0.1953
}
```

If you have the `jq` utility, it is a convenient way to pick things out:

```
jq '.steps[] | {command, status, seconds}' report.json
```

Chapter 4

Scripts

4.1 Structure

A script is a JSON object with four keys. Only two of them are required.

Key	Meaning
<code>project</code>	The project file to load. Required.
<code>steps</code>	The commands to carry out, as an array. Required.
<code>output</code>	Where to write the report. If absent, the report is printed to standard output.
<code>on_error</code>	"stop" (the default) abandons the run after a step fails; "continue" carries on with the remaining steps.
<code>working_folder</code>	Where analyses that write files of their own should put them. Defaults to the directory holding the script.

4.2 Steps run in order, and they share one project

`steps` is an array rather than an object, and that is deliberate: the order matters. All the steps act on one dataset, and several of them change it. Outlier removal, the organic matter and particle size correction, and the element selection commands all alter what the steps after them will see.

This is the main thing to understand about scripts. A script is not a list of independent jobs; it is a sequence, in the same way that a session in the interface is a sequence.

4.3 Steps

Each step is an object with a command and its arguments:

```
{
  "command": "Outlier",
  "arguments": { "Threshold": "3" }
}
```

The command names and the argument names are exactly those the dialogs use, including their capitalisation and spacing. `-describe` will tell you what a command accepts.

Argument values are written as text. Numbers and `true/false` are also accepted and converted, so both of these work:

```
"arguments": { "Number of samples": "1000", "Dissolve Chains": "true"  }  
"arguments": { "Number of samples": 1000,   "Dissolve Chains": true    }
```

4.4 You need only give what you want to change

Anything you leave out is filled in with the default the dialog would have shown. The report lists what was filled in, under `defaulted`, so the record of the run is complete even where the script was not.

This is worth relying on. Specify the two or three settings that matter for what you are doing and leave the rest alone; the run is still reproducible, because the report records the values that were actually used.

4.5 Paths

A path in a script that is not absolute is interpreted relative to the directory holding the script. A script and its project can therefore be moved or copied together, and sent to someone else, without editing.

Files that an analysis writes itself, such as the MCMC sample output, go to `working_folder`, which defaults to the script's directory.

4.6 Checking before running

Every step is checked before any of them runs: that the command exists, and that each argument is one the command accepts. If anything is wrong, nothing is carried out and the problems are listed:

```
error: The script was not run because of the following:  
  step 1: Unknown command 'Levenberg-Marquart'. Did you mean  
          'Levenberg-Marquardt'?  
  step 2: Command 'Levenberg-Marquardt' has no argument 'Sampel'.  
          Did you mean 'Sample'?
```

Checking everything first matters because of the previous section. A script that stopped halfway through because of a spelling mistake in the last step would leave the project in a state you did not ask for.

Chapter 5

Configuring the project

The analyses work on whatever the project file was saved with: which constituents are included, which samples are in use, what each constituent is, and which group holds the targets. In the interface those are set in tables and dialogs rather than through the analysis menus.

The four commands in this chapter set them from a script. Where one of them disagrees with the project file, the command wins: running one is how a script says it wants something other than what was saved.

They are what make a script useful for more than replaying a saved configuration. Comparing tracer sets, or correcting an isotope's settings, is a matter of one step.

5.1 set-target-group

Chooses which group holds the target samples.

Argument	Value	Default	Notes
group	name	—	a group name in the project

```
{ "command": "set-target-group", "arguments": { "group": "Mixture" } }
```

5.2 set-elements

Chooses which constituents take part.

Argument	Value	Default	Notes
only	name or list	—	include exactly these, exclude everything else
include	name or list	—	include these, leave the rest alone
exclude	name or list	—	exclude these, leave the rest alone

only is applied first, then **include**, then **exclude**. Every name is checked before anything is changed, so a misspelling cannot leave the project half configured.

```
{ "command": "set-elements",  
  "arguments": { "only": ["Aluminum", "Barium", "Lead", "Zinc"] } }
```

5.3 set-samples

Chooses which samples in a group take part.

Argument	Value	Default	Notes
group	name	—	required
only	name or list	—	include exactly these within the group
include	name or list	—	include these
exclude	name or list	—	exclude these

```
{ "command": "set-samples",
  "arguments": { "group": "Bank", "exclude": ["B7", "B12"] } }
```

5.4 set-element-role

Sets what a constituent is, and for an isotope what it is measured against.

Argument	Value	Default	Notes
element	name	—	required
role	name	—	Element, Isotope, ParticleSize, OM or DoNotInclude
base element	name	—	for an isotope: the constituent supplying its concentration
standard ratio	number	—	for an isotope: the standard isotopic ratio

```
{ "command": "set-element-role",
  "arguments": { "element": "C13", "role": "Isotope",
                 "base element": "TOC", "standard ratio": "0.011113" } }
```

Isotopes need their base element to be part of the analysis. An isotope is measured as a delta value, and converting that to a concentration requires the concentration of the constituent it is referenced against. If the base element is not included, the conversion has nothing to work with.

A base element must also have the role **Element**. Constituents whose role is **OM** or **ParticleSize** are set aside before the analyses run, so an isotope referenced against organic carbon needs that constituent given the role **Element** and included:

```
{ "command": "set-element-role",
  "arguments": { "element": "TOC", "role": "Element" } },
{ "command": "set-elements", "arguments": { "include": ["TOC"] } }
```

Note that this also makes the base element a fingerprinting tracer in its own right, which changes the apportionment. That is a modelling decision, so make it deliberately.

Chapter 6

Analysis commands

Every command the software accepts is listed here, with its arguments, the values each one takes, and its default. The descriptions are the same ones the dialogs show.

The **Value** column says what kind of thing to write in the script. Where it says “name”, the accepted names come from the loaded project — a target sample, a source group or a constituent — and the **Notes** column says which.

6.0.1 ANOVA

This tool performs analysis of variance. The resulting p-value indicates the power of a certain element in discriminating between all source groups.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
Log Transformation	true / false	"false"	—
Modify the included elements based on the results	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
P-value threshold	number or text	"0.05"	—
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.2 AutoSelect

The **Auto-select** tool selects a minimum number of elements with discriminatory power to differentiate between all source pair combinations. It performs a *t-test* between each pair of source groups and select the elements with the lowest p-values from each pair. The number of elements chosen from each pair is indicated in the box labeled 'Number of elements from each pair'

Argument	Value	Default	Notes
Include Isotopes	true / false	"false"	—
Modify the included elements based on the results	true / false	"false"	—
Number of elements from each pair	whole number	"3"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all

6.0.3 BoxCox

This tool calculates the optimal lambda value in the Box-Cox transformation for the elements in the specified source group to achieve normality.

Argument	Value	Default	Notes
Source/Target group	name	—	a source group name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.4 Bracketing Analysis

This tool shows whether the elemental content of the elements in the specified target sample is outside of the range of the elemental contents in all source groups.

Argument	Value	Default	Notes
Correct based on size and organic matter	true / false	"false"	—
Sample	name	—	a target sample name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.5 Bracketing Analysis Batch

This tool shows whether the elemental content of the elements in target samples is outside of the range of the elemental contents in all source groups for all target samples.

Argument	Value	Default	Notes
Correct based on size and organic matter	true / false	"false"	—
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.6 CMB Bayesian

This tool performs Bayesian fingerprinting using Markov-Chain Monte Carlo algorithm for the specified target sample.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"false"	—
Dissolve Chains	true / false	"true"	—
Number of chains	whole number	"8"	—
Number of samples	whole number	"1000"	—
Sample	name	—	a target sample name
Samples File Name	file name	"MCMC_Output.txt"	—
Samples to be discarded (burnout)	whole number	"100"	—

6.0.7 CMB Bayesian-Batch

This tool performs Bayesian fingerprinting using Markov-Chain Monte Carlo algorithm for all target sample.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"false"	—
Dissolve Chains	true / false	"true"	—
Number of chains	whole number	"8"	—
Number of samples	whole number	"1000"	—
Samples File Name	file name	"MCMC_Output.txt"	—
Samples to be discarded (burnout)	whole number	"100"	—

6.0.8 CorMat

This tool generates the element correlation matrix for the chosen source group. To perform the correlation analysis on the corrected elemental profiles based on size and organic matter, select the target sample based on which the correction to be performed.

Argument	Value	Default	Notes
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Source/Target group	name	—	a source group name
Threshold	number or text	"0.75"	—
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.9 CovMat

This tool calculates the covariance matrix between the elemental contents of various elements for the specified source group.

Argument	Value	Default	Notes
Source/Target group	name	—	a source group name

6.0.10 DF

This tool shows fitted normal and log-normal distributions to the elemental content of the specified element in the specified source group.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
Constituent	name	—	a constituent name
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Source/Target group	name	—	a source group name
Use only selected samples	true / false	"false"	—

6.0.11 DFA

This tool performs Discriminant Function Analysis between the two source groups indicated. It provides the p-value, and the scatter matrix of projected elemental profiles based on the principle eigen vector of the discriminant hyper-plane.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Source/Target group I	name	—	a source group name
Source/Target group II	name	—	a source group name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.12 DFAM

This tool performs step-wise Discriminant Function Analysis between the two source groups indicated. It provides the S-value as a result of including various number of elements into the analysis, starting from the element that produces the maximum separation.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.13 DFAOnevsRest

This tool performs step-wise Discriminant Function Analysis between the two source groups indicated. It provides the S-value as a result of including various number of elements into the analysis, starting from the element that produces the maximum separation.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Source group	name	—	a source group name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.14 EDP

This tool calculates the measures of discriminant power of elements between the two specified source groups. The measures include within and between group difference ratio, discriminant fraction, and the T-test p-value.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
P-value threshold	number or text	"0.05"	—
Source/Target group I	name	—	a source group name
Source/Target group II	name	—	a source group name
Use only selected samples	true / false	"false"	—

6.0.15 EDPM

This tool calculates the measures of discriminant power of elements between all pairs of source groups. The measures include within and between group difference ratio, discriminant fraction, and the T-test p-value.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
Include target samples	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
P-value threshold	number or text	"0.05"	—
Use only selected samples	true / false	"false"	—

6.0.16 Error_Analysis

This tool uses Monte-Carlo simulation via bootstrapping to evaluate the effect of elimination of the indicated percentage of source samples on the fingerprinting results.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"true"	—
Number of realizations	whole number	"1000"	—
Percentage eliminated	whole number	"20"	—
Sample	name	—	a target sample name
Softmax transformation	true / false	"true"	—

6.0.17 GA

This tool performs sediment fingerprinting using the **Genetic Algorithm** optimization. The mean and standard deviations describing the true distribution of the elemental profiles of each element for each source are estimated as part of the maximization of the likelihood of observing the measured target sample elemental profile.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"false"	—
Cross-over probability	number or text	"1.0"	—
GA output file	file name	"GA_Output.txt"	—
Mutation probability	number or text	"0.02"	—
Number of Generations	whole number	"100"	—
Number of threads to be used	whole number	"8"	—
Numerical cross-over	true / false	"false"	—
Population	whole number	"100"	—
Sample	name	—	a target sample name
Shake coefficient	number or text	"0.05"	—
Shake coefficient reduction factor	number or text	"0.75"	—
Steepest Descent	true / false	"true"	—

6.0.18 GA (disregarding targets)

Argument	Value	Default	Notes
Cross-over probability	number or text	"1.0"	—
GA output file	file name	"GA_Output.txt"	—
Mutation probability	number or text	"0.02"	—
Number of Generations	whole number	"40"	—
Number of threads to be used	whole number	"8"	—
Numerical cross-over	true / false	"false"	—
Population	whole number	"40"	—
Shake coefficient	number or text	"0.05"	—
Shake coefficient reduction factor	number or text	"0.75"	—

6.0.19 GA (fixed elemental contribution)

This tool performs sediment fingerprinting using the **Genetic Algorithm** optimization. The mean and standard deviations describing the true distribution of the elemental profiles of each element for each source are estimated by direct calculation based on the samples.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"false"	—
Cross-over probability	number or text	"1.0"	—
GA output file	file name	"GA_Output.txt"	—
Mutation probability	number or text	"0.02"	—
Number of Generations	whole number	"40"	—
Number of threads to be used	whole number	"8"	—
Numerical cross-over	true / false	"false"	—
Population	whole number	"40"	—
Sample	name	—	a target sample name
Shake coefficient	number or text	"0.05"	—
Shake coefficient reduction factor	number or text	"0.75"	—

6.0.20 KS

This tool provides visualization of Kolomogrov-Smirnov normality test for the indicated source group. It can be used to evaluate how close the elemental content of all elements are to normal or log-normal distribution for the specified source group.

Argument	Value	Default	Notes
Distribution	name	—	one of Normal, Lognormal
Source/Target group	name	—	a source group name

6.0.21 KS-individual

This tool provides visualization of Kolomogrov-Smirnov normality test for the indicated source group. It can be used to evaluate how close the elemental content of a given element is to normal or log-normal distribution for the specified source group.

Argument	Value	Default	Notes
Constituent	name	—	a constituent name

Argument	Value	Default	Notes
Distribution	name	—	one of Normal, Lognormal
Source/Target group	name	—	a source group name

6.0.22 Levenberg-Marquardt

This tool uses the Levenberg-Marquardt optimization algorithm to perform sediment fingerprinting by minimizing the RMSE between the modeled and observed elemental profiles of the target sample. The mean and standard deviations representing the true distribution of elemental profiles for each element and source are directly calculated from the samples.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"true"	—
Sample	name	—	a target sample name
Softmax transformation	true / false	"true"	—

6.0.23 Levenberg-Marquardt-Batch

This tool performs sediment fingerprinting using the **Levenberg-Marquardt** optimization algorithm on all target samples. The mean and standard deviations describing the true distribution of the elemental profiles of each element for each source are estimated by direct calculation based on the samples. This tool uses the Levenberg-Marquardt optimization algorithm to perform sediment fingerprinting by minimizing the RMSE between the modeled and observed elemental profiles of the target sample. The mean and standard deviations representing the true distribution of elemental profiles for each element and source are directly calculated from the samples.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"true"	—
Softmax transformation	true / false	"true"	—

6.0.24 MLR

This tool performs the regression analysis for organic and size correction on all source groups.

Argument	Value	Default	Notes
Equation	name	—	one of Linear, Power
Organic Matter constituent	name	—	a constituent name, or empty, or the organic matter constituent
P-value threshold	number or text	"0.05"	—
Particle Size constituent	name	—	a constituent name, or empty, or the particle size constituent
Use only selected samples	true / false	"false"	—

6.0.25 OM-Size Correct

This tool shows the corrected elemental profiles of source groups after adjusting based on size and organic matter. The correction is done based on the size and organic matter associated

with the selected target. The regressions as a result of the last organic and size correction will be used to perform the adjustment.

Argument	Value	Default	Notes
Sample	name	—	a target sample name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.26 Outlier

This tool conducts outlier analysis on the elemental profiles for the specified source group. The analysis begins by applying a Box-Cox transformation to each elemental content, followed by calculating the standardized value using the formula $(x - \text{mean}) / \text{std}$. Any absolute values exceeding the user-defined threshold are highlighted.

Argument	Value	Default	Notes
Source/Target group	name	—	a source group name
Threshold	number or text	"3"	—
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.27 SDFA

This tool performs step-wise Discriminant Function Analysis between the two source groups indicated. It provides the S-value as a result of including various number of elements into the analysis, starting from the element that produces the maximum separation.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Source/Target group I	name	—	a source group name
Source/Target group II	name	—	a source group name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.28 SDFAM

This tool performs step-wise Discriminant Function Analysis between the two source groups indicated. It provides the S-value as a result of including various number of elements into the analysis, starting from the element that produces the maximum separation.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
Modify the included elements based on the results	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.29 SDFAOnewsRest

This tool performs step-wise Discriminant Function Analysis between the two source groups indicated. It provides the S-value as a result of including various number of elements into the analysis, starting from the element that produces the maximum separation.

Argument	Value	Default	Notes
Box-cox transformation	true / false	"false"	—
OM and Size Correct based on target sample	name	—	a target sample name, or empty for all
Source group	name	—	a source group name
Use only selected elements	true / false	"false"	—
Use only selected samples	true / false	"false"	—

6.0.30 Source_Verify

This tool evaluates whether the finger-printing algorithm is capable of correctly infer the source samples based on their elemental profiles. The analysis is done by treating individual source samples from the specified source group as target samples and the use the Levenberg-Marquardt optimization to infer the contribution of sources into it.

Argument	Value	Default	Notes
Apply size and organic matter correction	true / false	"true"	—
Softmax transformation	true / false	"true"	—
Source Group	name	—	a source group name

6.0.31 Test CMB Bayesian

Argument	Value	Default	Notes
Number of chains	whole number	"8"	—
Number of samples	whole number	"1000"	—
Samples to be discarded (burnout)	whole number	"100"	—
samples_file_name	file name	"MCMC_Output.txt"	—

Chapter 7

The report

A run produces one JSON file describing everything that happened. It is both the result and the record: it states which project was used, what each step was asked to do, what it did, and what came out.

7.1 What is in it

Key	Meaning
<code>script</code>	The script that was run.
<code>project</code>	The project that was loaded.
<code>working_folder</code>	Where analyses wrote files of their own.
<code>started, finished</code>	When the run began and ended.
<code>succeeded</code>	Whether every step succeeded.
<code>steps</code>	One entry per step, in the order they ran.
<code>stopped_at_step</code>	Present only if the run was abandoned after a failure.

And within each step:

Key	Meaning
<code>step, command</code>	Position in the script, and what was run.
<code>arguments</code>	The arguments as given.
<code>defaulted</code>	Arguments that were not given and were filled in.
<code>status</code>	<code>succeeded</code> or <code>failed</code> .
<code>seconds</code>	How long it took.
<code>warnings</code>	Anything the analysis reported, such as a reason it declined to run.
<code>results</code>	What it produced.
<code>changes</code>	For a setup command, what it altered.

7.2 Results

Results are numbered in the order the analysis produced them, as they are in the interface: 1:Contributions, 2:Modeled Elemental Profile, and so on. What each contains depends on the analysis; a contribution is a figure per source, while a credible interval is a low and high value per source.

7.3 Exit status

The program exits with status 0 only if every step succeeded, and 1 otherwise. A script can rely on this:

```
sedst3-cli run.json && echo "all steps succeeded"
```

Chapter 8

Options

Option	Meaning
<code>-output <i>file</i></code>	Where to write the report, overriding the script.
<code>-quiet</code>	Do not print progress or announce each step. Warnings are still printed, and the report is unchanged.
<code>-resources <i>dir</i></code>	Where to find <code>forms_structures.json</code> .
<code>-list-commands</code>	List every command and stop.
<code>-describe <i>cmd</i></code>	Describe one command and stop.
<code>-help</code>	Print usage and stop.

Chapter 9

Worked examples

9.1 A deterministic workflow

The sequence the interface presents under *Deterministic Sediment Fingerprinting*, written down. Note that the steps build on one another: the correction has to happen before the fingerprinting that relies on it.

```
{
  "project": "my-project.cmb",
  "output": "deterministic.json",
  "on_error": "stop",
  "steps": [
    { "command": "Outlier",
      "arguments": { "Threshold": "3" } },

    { "command": "MLR",
      "arguments": { "Organic Matter constituent": "TOC",
                    "Particle Size constituent": "D50" } },

    { "command": "Bracketing Analysis Batch",
      "arguments": {} },

    { "command": "SDFAM",
      "arguments": { "Use only selected elements": "true" } },

    { "command": "Levenberg-Marquardt-Batch",
      "arguments": { "Apply size and organic matter correction": "true" } }
  ]
}
```

9.2 A Bayesian run

```
{
  "project": "my-project.cmb",
  "output": "bayesian.json",
  "steps": [
    { "command": "CMB Bayesian-Batch",
      "arguments": {
        "Number of samples": "1000",
        "Number of chains": "8",
        "Samples to be discarded (burnout)": "100",
        "Apply size and organic matter correction": "false"
      }
    }
  ]
}
```

```
    } }  
  ]  
}
```

This writes a good deal besides the report: for each target sample, a folder containing the chains, the posterior distributions, the credible intervals and the predicted concentrations. They go under `working_folder`.

9.3 Comparing two tracer sets

The case the command line build is best at. Two scripts differing in one line, each writing its own report, and the two compared afterwards.

```
{  
  "project": "my-project.cmb",  
  "output": "with-isotopes.json",  
  "steps": [  
    { "command": "set-elements",  
      "arguments": { "only": ["Aluminum","Barium","Lead","Zinc",  
                             "TOC","N","C13","N15"] } },  
    { "command": "Levenberg-Marquardt-Batch",  
      "arguments": { "Apply size and organic matter correction": "false" } }  
  ]  
}
```

and the same with the isotopes left out:

```
{ "command": "set-elements",  
  "arguments": { "only": ["Aluminum","Barium","Lead","Zinc"] } },
```

Because the only difference between the two runs is recorded in the scripts, the comparison means something, and it can be repeated.

Chapter 10

When something goes wrong

10.1 The script was not run

Every step is checked first, so this means a command or argument name was not recognised. The message names the step and suggests the closest accepted name. Names must match the dialogs exactly, including spaces and capitalisation; `-describe` is the authority.

10.2 A step failed

The step's entry in the report says so, and its **warnings** usually say why. The common ones:

Perform Organic Matter and Size Correction first

An analysis was asked to apply the correction, but the correction has not been carried out. Add an MLR step before it, or set `"Apply size and organic matter correction"` to `"false"`.

Zero or negative values for element ...

A constituent has values a log-normal distribution cannot represent. Exclude it with `set-elements`, or exclude the samples responsible with `set-samples`.

Singular matrix in within group scatter matrix

A discriminant analysis was given constituents that do not carry independent information. Reduce the set.

10.3 Nothing seems to be included in the analysis

Projects saved by older versions do not record which constituents are included, and they load with none of them selected. Put a `set-elements` step at the start of the script to say explicitly what should be used.

10.4 Warnings about a singular system

Messages of the form

```
warning: solve(): system is singular (rcond: 1.0e-19);  
attempting approx solution
```

come from the matrix library during the Levenberg-Marquardt fit. They are normally emitted in the last few iterations, after the fit has converged and the remaining steps are negligible, and in that case the result is sound — check that the contributions are sensible and sum to one.

If they appear from the beginning of a fit rather than at the end, the parameters are not identifiable from the data: there are too few informative tracers for the number of sources, or the tracers chosen do not distinguish them. Reducing the source count or choosing better tracers is the remedy, not running longer.

10.5 It cannot find the command definitions

The program warns at startup if it cannot find `resources/forms_structures.json`, and the message lists everywhere it looked. Set `SEDSAT3_RESOURCES` or pass `-resources`. Without it, scripts are not checked and omitted arguments are not filled in, so analyses may fail in confusing ways.

Index

ANOVA, 11
AutoSelect, 11

BoxCox, 12
Bracketing Analysis, 12
Bracketing Analysis Batch, 12

CMB Bayesian, 12
CMB Bayesian-Batch, 12
CorMat, 13
CovMat, 13

DF, 13
DFA, 13
DFAM, 14
DFAOnevsRest, 14

EDP, 14
EDPM, 15
Error_Analysis, 15

GA, 15
GA (disregarding targets), 15
GA (fixed elemental contribution), 16

KS, 16
KS-individual, 16

Levenberg-Marquardt, 17
Levenberg-Marquardt-Batch, 17

MLR, 17

OM-Size Correct, 17
Outlier, 18

SDFA, 18
SDFAM, 18
SDFAOnevsRest, 19
Source_Verify, 19

Test CMB Bayesian, 19